DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 11
Containers e Dockerfile multi-stage

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Até aqui a API do ServiceHub roda no seu computador, com o seu Java, o seu Maven e as suas variáveis. Nesta semana ela ganha uma imagem própria: um pacote que leva a aplicação e o ambiente de que ela precisa, e que roda do mesmo jeito no notebook, no servidor de integração contínua e em produção. Você vai entender o que é um container por dentro, por que ele não é uma máquina virtual, como as camadas de uma imagem funcionam e o que torna uma imagem pequena e segura. No fim, escreve (com ajuda da IA, revisando tudo) o Dockerfile multi-stage da sua API e prova que ele funciona.

Conteúdo

1. Objetivos da semana
2. "Na minha máquina funciona"
3. Virtualização e containers
4. Namespaces e cgroups
5. Imagem, camada, registro, tag e digest
6. Camadas, copy-on-write e cache
7. Ciclo de vida e sinais
8. Escolhendo a imagem base
9. Princípios de uma imagem segura
10. The Twelve-Factor App
11. O Dockerfile do ServiceHub
12. Comandos do dia a dia
13. Construindo com IA
14. Glossário
15. Exercícios
16. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. diferenciar máquina virtual de container, explicando o papel do hipervisor, do kernel compartilhado, dos namespaces e dos cgroups;
2. usar com precisão os termos imagem, camada, registro, container, tag e digest, e explicar o copy-on-write e o cache de build;
3. escolher uma imagem base e aplicar os princípios de imagem segura: não-root, menor superfície e nenhum segredo dentro;
4. ler e escrever um Dockerfile multi-stage para uma API Spring Boot com Java 21, com HEALTHCHECK, fuso horário e limite de memória da JVM;
5. construir, rodar e inspecionar a imagem com `docker build`, `run`, `ps`, `logs` e `stop`, e revisar criticamente um Dockerfile gerado por IA.

2. "Na minha máquina funciona"

↑ conteúdo

Toda equipe de software já ouviu a frase. Ela quer dizer uma coisa precisa: o **ambiente** faz parte do software, e ninguém o entregou junto com o código. O programa depende de uma versão do Java, de variáveis de ambiente, do fuso horário do sistema, de bibliotecas instaladas, de uma porta livre. Cada diferença entre a máquina de quem desenvolve e o servidor é um defeito esperando para aparecer.

No ServiceHub: um colega tem Java 17 e o projeto exige 21; o PostgreSQL local está em outra versão; as datas das ordens aparecem com três horas de diferença porque o servidor está em UTC. Nenhum desses erros está no código Java.

A resposta da indústria foi **empacotar a aplicação junto com o ambiente de que ela precisa** e entregar esse pacote, sempre o mesmo, do notebook à produção. Esse pacote é a **imagem**, e a aplicação rodando a partir dela é o **container**.

ANALOGIA

Antes dos contêineres de carga, cada navio era carregado peça por peça, com sacos, caixas e barris de tamanhos diferentes, e cada porto tinha seu jeito de manusear. O contêiner padronizou a embalagem: o guindaste não precisa saber o que vai dentro. O container de software faz o mesmo: o servidor não precisa saber se dentro há Java, Python ou Node; ele só sabe rodar containers.

3. Virtualização e containers

↑ conteúdo

Há duas formas principais de isolar aplicações numa mesma máquina física: **virtualizar o hardware** (fingir, por software, que há vários computadores) ou **isolar processos** (fazer cada aplicação acreditar que está sozinha num sistema operacional que, na verdade, é compartilhado).

Máquinas virtuais e o hipervisor

Uma **máquina virtual** (VM) é um computador simulado, com CPU, memória, disco e placa de rede virtuais. Dentro dela roda um **sistema operacional convidado** completo, com o seu próprio kernel. Quem cria e executa as VMs, dividindo os recursos físicos entre elas, é o **hipervisor**.

Há dois tipos de hipervisor. O de **tipo 1** (nativo) roda direto sobre o hardware, sem sistema operacional por baixo; é o modelo dos data centers e das nuvens (exemplos: Hyper-V, KVM). O de **tipo 2** (hospedado) roda como um programa comum sobre um sistema operacional já instalado, como o VirtualBox num computador de desenvolvimento.

Containers e o kernel compartilhado

Um **container** não simula hardware nem traz kernel próprio. Ele é um processo comum do sistema operacional hospedeiro, isolado por recursos do próprio kernel Linux. Todos os containers de uma máquina usam **o mesmo kernel**, o do hospedeiro. O que cada container leva é só a aplicação e as bibliotecas e arquivos de que ela precisa.



Cada VM traz o seu sistema operacional e o seu kernel; os containers compartilham o kernel do hospedeiro.

Critério	Máquina virtual	Container
O que leva	Sistema operacional completo	Só a aplicação e suas bibliotecas
Kernel	Próprio, sobre um hipervisor	Compartilha o do hospedeiro
Isolamento	Forte, no nível do hardware virtual	Namespaces e cgroups do Linux
Inicialização	Minutos (é preciso dar boot num SO)	Segundos (é preciso iniciar um processo)

Critério	Máquina virtual	Container
Quando usar	Rodar outro sistema operacional inteiro; isolamento forte entre clientes	Empacotar, distribuir e escalar serviços

As duas tecnologias não competem: na nuvem, é comum rodar containers dentro de VMs.

CONFUSÃO COMUM

"Container é uma VM leve." Não é. Uma VM tem kernel próprio; um container é um processo do hospedeiro. Consequência prática: como o kernel é compartilhado, não dá para rodar um container de Linux diretamente num kernel Windows. Por isso o **Docker Desktop**, no Windows e no macOS, roda uma VM Linux leve por baixo (no Windows, via WSL 2, que usa o Hyper-V), e os seus containers vivem dentro dela.

4. Namespaces e cgroups

↑ conteúdo

Se o container é só um processo, de onde vem o isolamento? De dois recursos do kernel Linux que o Docker combina de forma fácil de usar: **namespaces**, que controlam o que o processo *vê*, e **cgroups**, que controlam quanto ele *usa*.

Namespaces: o que o processo enxerga

Um **namespace** dá a um grupo de processos uma visão própria de um recurso global do sistema. Há namespaces de **PID** (dentro do container, a aplicação é o processo 1 e só enxerga os processos do próprio container), de **rede** (interfaces, IP e portas próprios: a 8080 de um container não conflita com a 8080 de outro), de **mount** (a árvore de diretórios vem da imagem, não do hospedeiro), **UTS** (nome do host próprio), de **usuários** e de **IPC** (comunicação entre processos).

cgroups: quanto o processo pode usar

Os **control groups** (cgroups) limitam e medem o consumo de CPU, memória e entrada e saída de um grupo de processos. Quando você roda `docker run --memory=512m`, o Docker cria um limite de cgroup. Se o processo tentar passar desse limite, o kernel o encerra (o famoso *OOM kill*, de *out of memory*), e o container sai com código 137.

Isso importa muito para Java. A JVM reserva memória para o **heap** (onde ficam os objetos) com base na memória que ela enxerga. Versões modernas da JVM leem o limite do cgroup, mas, por padrão, usam só um quarto dele para o heap. A opção `-XX:MaxRAMPercentage=75.0`, usada no Dockerfile do ServiceHub, diz à JVM para usar até 75% do limite do container, deixando o resto para pilhas das threads, metaespaço e o próprio sistema.

PARA LEMBRAR

Um container não é uma máquina pequena: é um **processo comum do Linux** com namespaces, cgroups e um sistema de arquivos próprio. Com a API rodando, `docker top` mostra o processo `java` visto pelo hospedeiro, com outro número de PID. Dentro do container, o mesmo processo é o PID 1. E como o kernel é compartilhado, isolamento não é segurança total: por isso a seção 9 insiste em rodar como usuário comum.

5. Imagem, camada, registro, tag e digest

↑ conteúdo

O vocabulário de containers é pequeno, mas precisa ser usado com precisão. Muita dúvida de depuração vem de confundir imagem com container.

Termo	Definição	No ServiceHub
Imagem	Pacote **só de leitura** com os arquivos e os metadados (comando de início, variáveis, porta documentada) da aplicação.	`servicehub-api`, gerada pelo `docker build`
Camada	A diferença de arquivos gerada por uma instrução do Dockerfile. A imagem é uma pilha de camadas.	A camada do `app.jar` muda a cada build
Container	Instância em execução de uma imagem, mais uma camada gravável só dele.	O processo `java` da API rodando
Registro	Servidor que guarda e distribui imagens.	Docker Hub, de onde vêm `maven` e `eclipse-temurin`
Tag	Nome legível e **móvel** de uma versão da imagem.	`21-jre-alpine`, `latest`
Digest	Hash `sha256` do conteúdo: identidade **imutável** da imagem.	`eclipse-temurin@sha256:…`

A relação entre imagem e container é a mesma entre classe e objeto: uma imagem, muitos containers. O que se grava dentro de um container fica na camada gravável dele e some quando ele é removido. Para mudar a aplicação, não se "edita o container": gera-se uma imagem nova e sobe-se um container novo a partir dela.

Tag é apelido; digest é identidade

O nome completo de uma imagem é `registro/repositório:tag`. Quando você escreve só `eclipse-temurin:21-jre-alpine`, o Docker completa com o registro padrão, o Docker Hub (`docker.io/library/eclipse-temurin:21-jre-alpine`). Quando omite a tag, ele usa `latest`.

A imagem pode ser referenciada por tag (`FROM eclipse-temurin:21-jre-alpine`) ou por digest (`FROM eclipse-temurin:21-jre-alpine@sha256:<hash>`); `docker images --digests` mostra os

digests locais. Fixar por **tag de versão** é fácil de ler, e cada novo build recebe as correções publicadas sob a mesma tag; em troca, o mesmo nome pode dar imagens diferentes em dias diferentes. Fixar por **digest** dá build reprodutível, bit a bit; em troca, atualizar passa a ser tarefa sua.

CONFUSÃO COMUM

`latest` não quer dizer "a versão mais nova". É só a tag usada quando nenhuma é informada. Uma imagem antiga pode estar marcada como `latest`, e um `FROM algo:latest` pode mudar de versão principal sem aviso. O ServiceHub usa tags de versão fixa (`maven:3.9-…`, `21-jre-alpine`): estável o bastante, e ainda recebe correções.

6. Camadas, copy-on-write e cache

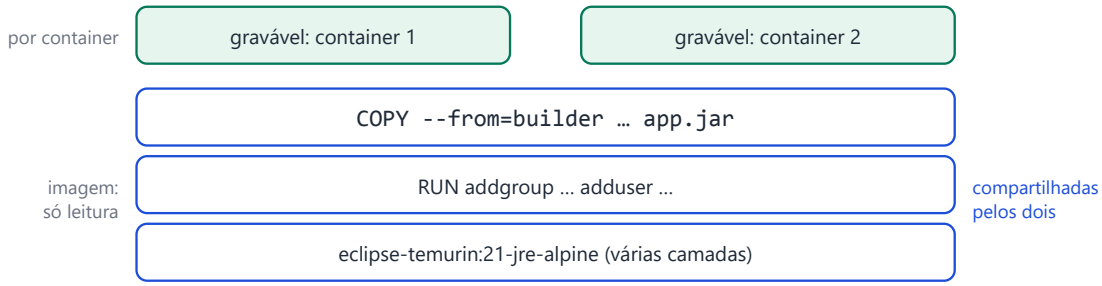
↑ conteúdo

Sistema de arquivos em camadas

Cada instrução do Dockerfile que mexe em arquivos (`FROM`, `COPY`, `RUN`) gera uma camada. O motor de containers sobrepõe as camadas e mostra ao processo um único sistema de arquivos, como folhas de transparência empilhadas. No Linux, o driver de armazenamento padrão do Docker para isso é o **overlay2**. Instruções como `ENV`, `EXPOSE`, `USER` e `HEALTHCHECK` não geram arquivos: viram **metadados** da imagem.

Copy-on-write

As camadas da imagem são só de leitura e compartilhadas por todos os containers criados a partir dela. Cada container ganha, por cima, uma camada gravável fina. A leitura de um arquivo vai direto à camada de baixo. Ao *alterar* um arquivo, ele é primeiro copiado para a camada gravável, e só a cópia muda. Daí o nome: copiar ao escrever.



Dois containers da mesma imagem: as camadas de baixo são compartilhadas; cada um tem só a sua camada gravável.

Consequências: dez containers da mesma imagem ocupam o espaço da imagem uma vez só; subir um container é rápido porque nada é copiado; e o que o container grava some quando ele é removido, por isso o banco guarda dados num **volume** (Semana 12). `docker history servicehub-api` lista as camadas e o tamanho de cada uma.

Cache de build: a ordem importa

Ao reconstruir, o Docker reaproveita uma camada se a instrução e os arquivos que ela usa não mudaram. Mas basta uma camada mudar para **todas as que vêm depois** serem refeitas. A regra que decorre disso: o que muda pouco vai em cima, o que muda sempre vai embaixo. Num projeto Maven, o `pom.xml` muda raramente; o código em `src/` muda a cada commit.

Sem cache útil

```
FROM maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY . .
RUN mvn package
# mudou uma linha de Java?
# baixa todas as dependências de novo
```

Dependências primeiro

```
FROM maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q package -DskipTests
# só refaz a partir do COPY src
```

Enquanto o `pom.xml` não muda, a camada com todas as dependências vem pronta do cache, e o build cai de minutos para segundos. A saída do `docker build` marca como `CACHED` as etapas reaproveitadas. Para forçar um build do zero, use `--no-cache`.

7. Ciclo de vida e sinais

↑ conteúdo

Um container vive enquanto o seu **processo principal** vive. Se a JVM termina, por erro ou por ordem, o container para.

Os estados são: **created** (existe, com camada gravável, mas o processo não começou), **running** (o processo principal executa), **paused** (processos congelados por `docker pause`), **exited** (o processo terminou; camada gravável, log e código de saída continuam lá) e, depois de `docker rm`, **removido**: a camada gravável é apagada para sempre.

`docker run` faz `create` e `start` de uma vez; `docker ps -a` lista também os parados, com o código de saída; `--rm` remove o container ao sair; `--restart` faz o Docker subi-lo de novo quando cai.

SIGTERM, SIGKILL e o encerramento gracioso

No Linux, um **sinal** é uma notificação que o sistema envia a um processo. Dois importam aqui. O **SIGTERM** é um pedido educado: "termine, por favor". O processo pode capturá-lo, terminar as requisições em andamento, fechar conexões com o banco e sair. O **SIGKILL** não pode ser capturado: o kernel mata o processo na hora.

`docker stop` envia SIGTERM ao processo principal e espera um prazo (10 segundos, por padrão). Se o processo não sair, envia SIGKILL. Uma API que recebe o SIGTERM encerra de forma **graciosa**:

o Spring Boot fecha o contexto, para de aceitar requisições novas e libera o pool de conexões. Uma API que nunca recebe o SIGTERM morre por SIGKILL no meio do que estava fazendo.

CÓDIGOS DE SAÍDA

Quando um processo morre por sinal, o código de saída costuma ser 128 mais o número do sinal: 143 para SIGTERM (15) e 137 para SIGKILL (9). Ver 137 no `docker ps -a` indica morte forçada: `docker stop` sem resposta ou falta de memória. Se o sinal chega à JVM depende da forma do `ENTRYPOINT` (seção 10).

8. Escolhendo a imagem base

↑ conteúdo

Todo Dockerfile começa com `FROM`, que escolhe a **imagem base**: o sistema de arquivos sobre o qual a aplicação será colocada. Não existe escolha universal; é uma troca entre tamanho, superfície de ataque e conveniência.

Tipo	O que traz	Ganha	Paga
Distro completa	Sistema inteiro (Ubuntu, Debian), com shell e gerenciador de pacotes	Tudo à mão para depurar e instalar	Maior, com mais pacotes que podem ter falhas
Slim	A mesma distro sem documentação e utilitários extras	Menor, ainda com glibc e apt	Pode faltar ferramenta que um script usa
Alpine	Distro mínima com **musl libc** e BusyBox	Muito pequena; tem shell e `wget`	musl difere da glibc; biblioteca nativa compilada para glibc pode quebrar
Distroless	Só o runtime e suas dependências; sem shell nem gerenciador	Menor superfície de ataque	Sem shell para depurar nem para um HEALTHCHECK com `wget`

A **libc** é a biblioteca padrão do C, usada por quase todo programa nativo. Uma API Java pura não percebe a diferença entre glibc e musl, porque a JVM da imagem já foi compilada para a base certa; se o projeto depender de biblioteca nativa compilada só para glibc, prefira Debian ou Ubuntu. Outra decisão é **JDK** (*Java Development Kit*, com compilador) ou **JRE** (*Java Runtime Environment*, só para executar): compila-se com o JDK; roda-se com o JRE.

NO SERVICEHUB

O estágio de execução usa `eclipse-temurin:21-jre-alpine`: imagem oficial da distribuição Eclipse Temurin do OpenJDK, com só o JRE 21, sobre Alpine, e com o `wget` de que o HEALTHCHECK precisa.

ERRO COMUM EM CÓDIGO GERADO POR IA

Usar `FROM openjdk:17` ou `openjdk:21`. A imagem `openjdk` do Docker Hub foi descontinuada, e o projeto exige Java 21. Use `eclipse-temurin` com a tag da versão 21.

9. Princípios de uma imagem segura

↑ conteúdo

Tudo o que está numa imagem pode ser lido por quem a tiver, e tudo o que roda nela pode ser atacado. Quatro princípios resumem o cuidado:

Princípio	O que significa
Menor privilégio	Rode como usuário comum, não como root. Se a aplicação for invadida, o atacante herda as permissões do processo; quanto menos permissão, menor o estrago.
Menor superfície	Cada pacote a mais é uma vulnerabilidade possível. JRE em vez de JDK; nada de compilador, Maven ou código-fonte na imagem final.
Nenhum segredo dentro	Camadas são permanentes. Um `.env` copiado ou um `ENV` com senha fica no histórico da imagem, mesmo que uma camada seguinte apague o arquivo. Segredo entra só na hora de rodar.
Origem confiável	Imagens oficiais, versão fixada e reconstrução periódica para receber correções de segurança da base.

Um **scanner de vulnerabilidades** compara os pacotes da imagem com bases públicas de falhas conhecidas, costuma rodar no CI e completa os princípios, sem substituí-los.

10. The Twelve-Factor App

↑ conteúdo

Uma única imagem só serve para qualquer ambiente se a aplicação foi feita para isso. O **Twelve-Factor App** é uma metodologia publicada por engenheiros do Heroku para aplicações que rodam como serviço: não é norma formal, mas virou referência para aplicações em nuvem. Quatro fatores pesam mais em containers.

Fator III: configuração no ambiente

Configuração é tudo o que muda entre ambientes: URL do banco, credenciais, segredos, portas. Ela sai do código e vem de **variáveis de ambiente**. Um teste simples: o repositório poderia virar público agora sem expor nenhuma credencial? A imagem é construída uma vez e promovida de ambiente em ambiente; muda só a configuração. Se você precisa gerar uma imagem diferente para produção, a configuração está no lugar errado.

```
# application.properties do ServiceHub (trecho)
server.port=${SERVER_PORT:8080}
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5433/servicehub}
servicehub.jwt.secret=${JWT_SECRET:}
```

A sintaxe `${VAR:padrão}` do Spring lê a variável de ambiente e, se ela não existir, usa o padrão depois dos dois-pontos. O padrão serve só ao desenvolvimento. `JWT_SECRET` não tem padrão de propósito: sem ele, a API se recusa a subir.

Fatores VI e IX: sem estado e descartável

Sem estado (VI): nada que importa fica na memória ou no disco do processo; sessão e dados vão para serviços de apoio, como o banco. A API com JWT não guarda sessão (Semana 09). **Descartável (IX):** o processo sobe rápido e desliga de forma graciosa ao receber SIGTERM. Containers são tratados como gado, não como animais de estimação: o orquestrador mata e recria quando quiser, e isso só é seguro se o processo não guarda estado e encerra limpo.

Para o SIGTERM chegar à JVM, o `ENTRYPOINT` precisa estar na **forma exec** (lista JSON):

Forma exec

```
ENTRYPOINT ["java", "-jar", "app.jar"]
# java é o PID 1 e recebe
# o SIGTERM do docker stop
```

Forma shell

```
ENTRYPOINT java -jar app.jar
# /bin/sh -c é o PID 1; o sinal
# pode não chegar ao java
```

Na forma shell, o comando roda dentro de `/bin/sh -c`; o shell vira o processo principal e, em geral, não repassa o sinal. O container espera os 10 segundos e morre por SIGKILL.

Fator XI: logs como fluxo de eventos

A aplicação não cuida de arquivo de log: escreve cada evento no **stdout** (saída padrão), uma linha por evento. O ambiente captura o fluxo (no Docker, `docker logs`) e o encaminha a um coletor que indexa, busca e alerta, como ELK, Loki ou Datadog. Um `app.log` dentro do container some quando ele é removido, enche o disco e ninguém lê.

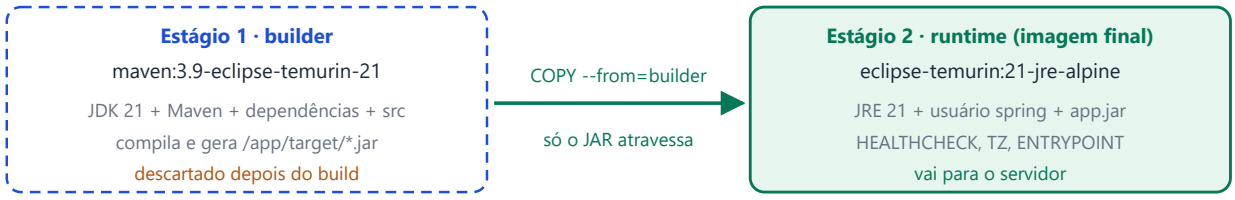
```
# application-prod.properties do ServiceHub
logging.structured.format.console=ecs   # JSON no console, com o requestId
```

Em desenvolvimento, o log é texto legível; no perfil `prod` (Semana 12), é JSON no formato ECS, que um coletor indexa sem parser. Nos dois casos vai para o console.

11. O Dockerfile do ServiceHub

↑ conteúdo

Para compilar a API você precisa de Maven e JDK. Para rodá-la, só de um JRE e do JAR. Um **Dockerfile multi-stage** separa as duas coisas num mesmo arquivo: cada `FROM` inicia um estágio, e o último estágio é a imagem final. Os anteriores servem de oficina e são descartados; só o que for copiado explicitamente com `COPY --from` atravessa.



Dois estágios num arquivo: a imagem final não leva código-fonte, Maven nem compilador.

Estágio 1: compilar

```
# ---------- Estágio 1: build (JDK + Maven) ----------
FROM maven:3.9-eclipse-temurin-21 AS builder
WORKDIR /app

# Dependências primeiro: esta camada fica em cache enquanto o pom.xml não mudar
COPY pom.xml .
RUN mvn -B -q dependency:go-offline

# Código-fonte e empacotamento (os testes rodam no CI e no mvnw verify, não aqui:
# precisariam de Docker dentro do Docker para o Testcontainers)
COPY src ./src
RUN mvn -B -q package -DskipTests -Djacoco.skip=true
```

`AS builder` dá nome ao estágio, para o `COPY --from` do estágio 2. `-B -q` é o modo *batch* (sem perguntas interativas) com log reduzido. Os testes são pulados porque o Testcontainers exigiria Docker dentro do Docker; quem prova a qualidade é o pipeline de CI (`mvnw verify`), que roda antes do build da imagem.

Estágio 2: executar

```
# ---------- Estágio 2: runtime (só JRE + JAR) ----------
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Usuário sem privilégios de root
RUN addgroup -S spring && adduser -S spring -G spring
USER spring
```

```
COPY --from=builder /app/target/*.jar app.jar

ENV TZ=America/Sao_Paulo
EXPOSE 8080
```

```
# O Docker marca o container como unhealthy se o Actuator parar de responder
HEALTHCHECK --interval=15s --timeout=5s --start-period=40s --retries=5 \
  CMD wget -qO- http://localhost:8080/actuator/health | grep -q '"status":"UP"' || exit 1

ENTRYPOINT ["java", \
  "-XX:MaxRAMPercentage=75.0", \
  "-Duser.timezone=America/Sao_Paulo", \
  "-jar", "app.jar"]
```

Linha	Por quê
`addgroup`, `adduser`, `USER`	Cria o grupo e o usuário de sistema `spring` (sintaxe do BusyBox do Alpine); daqui em diante, inclusive no `ENTRYPOINT`, nada roda como root.
`COPY --from=builder`	Traz do estágio 1 só o JAR, renomeado para `app.jar`.
`HEALTHCHECK`	A cada 15 s, consulta `/actuator/health`; espera até 5 s pela resposta; nos primeiros 40 s as falhas não contam, para dar tempo ao Spring e ao Flyway; após 5 falhas seguidas, o container fica `unhealthy`. O `docker ps` mostra `(health: starting)`, `(healthy)` ou `(unhealthy)`; o Docker só informa, não reinicia. Na Semana 12, o Compose usa esse estado para ordenar a subida.
`wget … | grep -q`	O `wget` existe no Alpine (o `curl` não vem por padrão). O `grep` exige o texto `"status":"UP"`, que o Actuator devolve a quem não está autenticado.
`TZ` e `-Duser.timezone`	Fuso do sistema e da JVM (que tem a sua própria base de fusos): datas das ordens no horário de Brasília.
`ENTRYPOINT [ … ]`	Forma exec: o `java` é o PID 1 e recebe o SIGTERM (seção 10). `MaxRAMPercentage=75.0`: heap até 75% do limite do container (seção 4). `EXPOSE` só documenta a porta (seção 12).

.dockerignore: o que nem chega ao build

Antes de executar o Dockerfile, o Docker envia ao motor o **contexto de build**: o diretório informado no comando (o `.` de `docker build .`). O arquivo `.dockerignore` lista o que fica de fora. O do ServiceHub (no arquivo real, um item por linha):

```
target/   .git/   .github/   .idea/   .vscode/   .mvn/   .superpowers/
docs/   frontend/   node_modules/   *.md   .env   .env.*
```

- **Contexto menor:** histórico do Git, documentação e o front-end de testes não interessam ao build; enviá-los só deixa tudo mais lento.
- **Nenhum segredo:** `.env` e variações nunca chegam a uma camada, mesmo que alguém escreva `COPY . .` no futuro.
- **Sem lixo local:** `target/` fica fora porque o JAR nasce dentro do estágio `builder`, e não no seu notebook.

12. Comandos do dia a dia

↑ conteúdo

```
docker build -t servicehub-api .            # gera a imagem a partir do Dockerfile
docker run -d --name api-aula -p 8090:8080 \
  -e JWT_SECRET=um-segredo-com-pelo-menos-32-bytes \
  -e DB_URL=jdbc:postgresql://host.docker.internal:5433/servicehub \
  servicehub-api
docker ps                                   # STATUS mostra (healthy) ou (unhealthy)
docker logs -f api-aula                     # segue o log do Spring
docker exec api-aula whoami                 # roda um comando dentro: deve dizer spring
docker stop api-aula                        # SIGTERM e encerramento gracioso
docker rm api-aula                          # remove o container parado
```

Opções mais usadas: `-t nome:tag` dá nome à imagem (sem tag, vale `latest`); `-d` roda em segundo plano; `--name` nomeia o container; `-p host:container` publica uma porta; `-e VAR=valor` define uma variável de ambiente (fator III); `--no-cache` faz o build sem reaproveitar camadas.

Dentro de um container, `localhost` é o próprio container. Para alcançar o PostgreSQL publicado no hospedeiro, o Docker Desktop oferece o nome `host.docker.internal`; na Semana 12, o Compose resolve o banco pelo nome do serviço.

PRIMEIRA DEPURAÇÃO

Sem `JWT_SECRET` (ou com menos de 32 bytes), a API do ServiceHub não sobe: a classe `JwtProperties` falha logo na inicialização. O container vai para `exited`, e `docker logs api-aula` mostra o motivo. Leia o log antes de perguntar a alguém (ou à IA).

EXPOSE não publica porta

ERRO COMUM

`EXPOSE 8080` só **documenta** a porta em que a aplicação escuta. Quem publica a porta no hospedeiro é o `-p host:container` do `docker run` (ou o `ports:` do Compose). Com `-p 8090:8080`, o Tomcat escuta na 8080 dentro do container, e você acessa `http://localhost:8090` no navegador. Sem `-p`, a porta fica inacessível de fora, com ou sem `EXPOSE`. Como cada container tem o seu namespace de rede, dois deles podem escutar na 8080 por dentro e ser publicados em 8090 e 8091 por fora.

13. Construindo com IA

↑ conteúdo

Um Dockerfile é curto e parece fácil de pedir à IA. O risco está aí: em vinte linhas cabem uma versão errada do Java, um processo como root e um segredo esquecido numa camada. Os quatro movimentos, nesta semana:

Movimento	Nesta semana
Especificar	Dois estágios, Java 21, build com `maven:3.9-eclipse-temurin-21`, runtime `eclipse-temurin:21-jre-alpine`, usuário não-root, HEALTHCHECK em `/actuator/health`, nenhum segredo na imagem.
Gerar	Pedir primeiro o plano (estágios, ordem das instruções, o que vai no `.dockerignore`) e só depois os dois arquivos.
Revisar	Versão da imagem base, `USER` antes do `ENTRYPOINT`, `pom.xml` antes de `src`, `ENTRYPOINT` em lista, `.env` no `.dockerignore`.
Provar	`docker ps` mostra `(healthy)`, `GET /api/v1/info` responde e `docker exec … whoami` devolve `spring`.

Prompt fraco e prompt bom

Fraco

```
cria um Dockerfile pra minha
API Spring Boot
```

Tende a vir com Java antigo, estágio único, processo como root, `COPY . .` no início e sem HEALTHCHECK.

Bom

Diz a stack e as versões, exige dois estágios com imagens nomeadas, lista as restrições de segurança (não-root, sem segredo, `.dockerignore`), diz como a configuração entra (variáveis de ambiente), pede o plano antes do código e define a prova: build, run e chamada a `/api/v1/info`.

O prompt desta semana

Continua o das semanas anteriores (a API já tem contrato, segurança, testes e Actuator). Troque o que estiver entre <>:

```
Contexto: somos uma equipe construindo a API ServiceHub em Java 21 e
Spring Boot 4.1 (Maven, com mvnw), PostgreSQL 16 e Flyway. A API já tem
Spring Security com JWT, testes de integração com Testcontainers e o
Actuator expondo /actuator/health como rota pública. Toda a configuração
vem de variáveis de ambiente com padrão de desenvolvimento no
application.properties (${DB_URL:...}, ${DB_USER:...}, ${DB_PASS:...},
${SERVER_PORT:8080}); JWT_SECRET é obrigatória (mínimo de 32 bytes) e não
tem padrão. Seguem o pom.xml e o application.properties: <cole aqui>.

Objetivo: criar na raiz do repositório um Dockerfile multi-stage e um
.dockerignore para empacotar a API numa imagem pequena e segura.

Restrições:
1. Estágio 1 "builder": FROM maven:3.9-eclipse-temurin-21. Copie primeiro
   só o pom.xml e rode mvn -B -q dependency:go-offline; depois copie src e
   rode mvn -B -q package -DskipTests -Djacoco.skip=true (os testes rodam
   no CI, pois o Testcontainers exigiria Docker dentro do Docker).
2. Estágio 2: FROM eclipse-temurin:21-jre-alpine, só o JRE e o JAR
   copiado com COPY --from=builder para /app/app.jar.
3. Crie o usuário e o grupo de sistema "spring" com addgroup/adduser do
   Alpine e use USER spring antes do ENTRYPOINT. Nada roda como root.
4. ENV TZ=America/Sao_Paulo e EXPOSE 8080.
5. HEALTHCHECK com wget (não curl) em http://localhost:8080/actuator/health
   exigindo "status":"UP"; interval 15s, timeout 5s, start-period 40s,
   retries 5.
6. ENTRYPOINT na forma exec (lista JSON) com java,
   -XX:MaxRAMPercentage=75.0, -Duser.timezone=America/Sao_Paulo, -jar app.jar.
7. NENHUM segredo na imagem: nada de ENV com senha ou JWT_SECRET, nada de
   COPY . . . O .dockerignore deve excluir pelo menos target/, .git/,
   .github/, .idea/, .vscode/, docs/, *.md, .env e .env.*.
8. NÃO use openjdk, JDK 17, imagem :latest, curl no healthcheck nem
   -XX:+UseContainerSupport. NÃO altere código Java nem o pom.xml.

Antes de escrever os arquivos, apresente um plano curto: os estágios, a
ordem das instruções e o motivo de cada uma, e o conteúdo do
.dockerignore. Espere minha confirmação. Depois entregue os dois
arquivos completos, com um comentário curto em cada bloco.

Critério de aceite: docker build -t servicehub-api . termina sem erro;
com um PostgreSQL acessível, docker run com -p 8090:8080 e -e JWT_SECRET
sobe o container; docker ps mostra (healthy); GET
http://localhost:8090/api/v1/info responde 200 com status "OK";
docker exec <container> whoami devolve spring; docker history não
mostra nenhum segredo.
```

O que a IA costuma errar nesta etapa

O que aparece no código gerado	O que o ServiceHub faz
Imagem `openjdk:17` ou JDK 17	Java 21: `eclipse-temurin` 21 no build e no runtime

O que aparece no código gerado	O que o ServiceHub faz
Nenhum `USER`: o processo roda como root	`addgroup`, `adduser` e `USER spring`
`COPY . .` antes de baixar dependências	`pom.xml` e `go-offline` primeiro, `src` depois
JDK completo na imagem final	Só o JRE no segundo estágio
HEALTHCHECK com `curl` sem conferir a imagem	`wget`, presente na imagem Alpine
`-XX:+UseContainerSupport` declarado	Já é o comportamento padrão da JVM; não precisa ser declarado
`ENTRYPOINT java -jar app.jar` (forma shell)	Forma exec, em lista

Checklist de revisão

☐ Há dois `FROM`, e o último usa `eclipse-temurin:21-jre-alpine`?

☐ O `pom.xml` é copiado e resolvido antes do `COPY src`?

☐ `USER spring` aparece antes do `ENTRYPOINT`?

☐ O HEALTHCHECK usa uma ferramenta que existe na imagem e consulta `/actuator/health`?

☐ O `ENTRYPOINT` está em forma de lista?

☐ Não há senha, token ou `JWT_SECRET` em nenhuma linha; `.env` está no `.dockerignore`?

Prova

Enquanto o Compose não chega (Semana 12), suba um PostgreSQL 16 provisório publicado na porta 5433, com as credenciais padrão do `application.properties`, e rode a API contra ele:

```
docker run -d --name pg-aula -p 5433:5432 -e POSTGRES_DB=servicehub \
  -e POSTGRES_USER=servicehub -e POSTGRES_PASSWORD=secret postgres:16-alpine

docker build -t servicehub-api .
docker run -d --name api-aula -p 8090:8080 \
  -e JWT_SECRET=um-segredo-de-teste-com-pelo-menos-32-bytes \
  -e DB_URL=jdbc:postgresql://host.docker.internal:5433/servicehub \
  servicehub-api

docker ps                              # aguarde (healthy)
curl http://localhost:8090/api/v1/info # no PowerShell, use curl.exe
docker exec api-aula whoami            # spring
```

A resposta esperada é um JSON como `{"status":"OK","version":"…","message":"ServiceHub API operacional"}`. Registre a saída do `docker ps` com `(healthy)` no README e, no PR, o pedido feito à IA e o que você corrigiu.

14. Glossário

↑ conteúdo

Container	Processo isolado do hospedeiro, criado a partir de uma imagem, com uma camada gravável própria.
Imagem	Pacote só de leitura com os arquivos e metadados de uma aplicação; pilha de camadas.
Camada	Diferença de arquivos gerada por uma instrução do Dockerfile.
Tag	Nome legível e móvel de uma versão de imagem.
Digest	Hash sha256 do conteúdo; identidade imutável de uma imagem.
Hipervisor	Software que cria e executa máquinas virtuais. Tipo 1 roda no hardware; tipo 2, sobre um SO.
Namespace	Recurso do kernel que dá a um processo uma visão própria de PIDs, rede, arquivos, hostname e usuários.
cgroup	Recurso do kernel que limita e mede CPU, memória e E/S de um grupo de processos.
Copy-on-write	Técnica em que um arquivo só é copiado para a camada gravável quando é alterado.
Multi-stage	Dockerfile com vários `FROM`; só o último estágio vira a imagem final.
HEALTHCHECK	Comando periódico que o Docker roda para marcar o container como healthy ou unhealthy.
SIGTERM	Sinal que pede ao processo para encerrar; pode ser tratado para um desligamento gracioso.
Contexto de build	Arquivos enviados ao motor no `docker build`, filtrados pelo `.dockerignore`.

15. Exercícios

↑ conteúdo

1. Explique, com suas palavras, por que a frase "na minha máquina funciona" indica um problema de ambiente e não de código. Dê dois exemplos que poderiam acontecer no ServiceHub.
2. Um container da API foi iniciado com `--memory=512m` e, sob carga, sai com código 137. Que recurso do kernel está envolvido, o que aconteceu e qual linha do Dockerfile do ServiceHub ajuda a evitar o problema?
3. Diferencie tag e digest. Por que `FROM eclipse-temurin:latest` é uma má escolha para o projeto?

4. Reordene as instruções abaixo para aproveitar o cache de build e explique a regra que você aplicou: `COPY src ./src`, `FROM maven:3.9-eclipse-temurin-21`, `RUN mvn -B -q package -DskipTests`, `COPY pom.xml .`, `RUN mvn -B -q dependency:go-offline`, `WORKDIR /app`.

5. Dois containers foram criados da mesma imagem. Um deles altera um arquivo de configuração dentro de si. O outro container vê a alteração? E se o primeiro for removido? Justifique com copy-on-write.

6. Encontre pelo menos quatro problemas no Dockerfile abaixo e corrija-os:

```
FROM openjdk:17
COPY . .
ENV JWT_SECRET=segredo123
RUN ./mvnw package
EXPOSE 8080
ENTRYPOINT java -jar target/app.jar
```

7. Um colega diz: "coloquei `EXPOSE 8090` no Dockerfile e mesmo assim não consigo acessar a API no navegador". Explique o erro e escreva o comando `docker run` correto para acessar a API em `localhost:8090`.

8. Relacione os fatores III, IX e XI do Twelve-Factor App com uma linha concreta do Dockerfile ou dos arquivos de propriedades do ServiceHub.

9. Escreva um prompt para pedir à IA que *revise* (e não reescreva) o Dockerfile da sua equipe, usando o checklist desta apostila. Diga o que você conferiria na resposta.

16. Para estudar

↑ conteúdo

Leitura disponível na Minha Biblioteca, pelo acesso institucional:

- MONTEIRO, E. R. et al. *DevOps*. Grupo A, 2021. Capítulo "Configuração e manutenção de contêineres usando o Docker".

Referências abertas:

- *The Twelve-Factor App*, fatores III (Configurações), VI (Processos), IX (Descartabilidade) e XI (Logs): `12factor.net` (há versão em português).
- Documentação oficial do Docker: *Multi-stage builds* (`docs.docker.com/build/building/multi-stage`) e *Dockerfile reference* (`docs.docker.com/reference/dockerfile`).
- Imagens oficiais usadas no projeto, no Docker Hub: `maven` e `eclipse-temurin`, com a lista de tags.
- No repositório do ServiceHub: `Dockerfile`, `.dockerignore`, `application.properties` e `application-prod.properties`.